Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Introducción a la programación 1

Manual Técnico Proyecto 1

202330209 Milton Efrén Ajxup Ramos

ÍNDICE

ÍNDICE —---------- 2

INTRODUCCIÓN —---------- 3

Diagrama Entidad Relación —---------- 4

Diagrama de Tablas —---------- 5

Manual Técnico —---------- 6

Servlets de Java —---------- 6

Peticiones de tipo get —---------- 6

Peticiones de tipo post —---------- 7

Peticiones de tipo put —---------- 8

Peticiones de tipo delete —---------- 8

Pagos —---------- 10

Subida de Archivos —---------- 11

INTRODUCCIÓN

Objetivo

Otorgar al usuario el soporte y ayuda necesaria para que pueda hacer una correcta ejecución de la aplicación de java del estudiante Milton, que en esta ocasión es el archivo del Proyecto 1 para el laboratorio de Introducción a la Programación y Computación 2.

Requerimientos

Tener una versión de java instalada

Tener disponible el archivo .war proporcionado por el estudiante.

Esta es una aplicación web que trabaja con un servidor de Angular para encender este primero desde la terminal se debe estar en la carpeta que contiene el archivo de angular del Proyecto y ejecutar el comando ng serve, esto activará el servidor y otro servidor de llamado Tomcat que se ejecuta dentro de un IDE compilando el archivo WAR del proyecto, así que hay que tener ambos instalados, para ejecutar el archivo, se accede en la web a http://localhost:4200, esto llevará al proyecto listo para ejecutarse.

Diagrama Entidad Relación

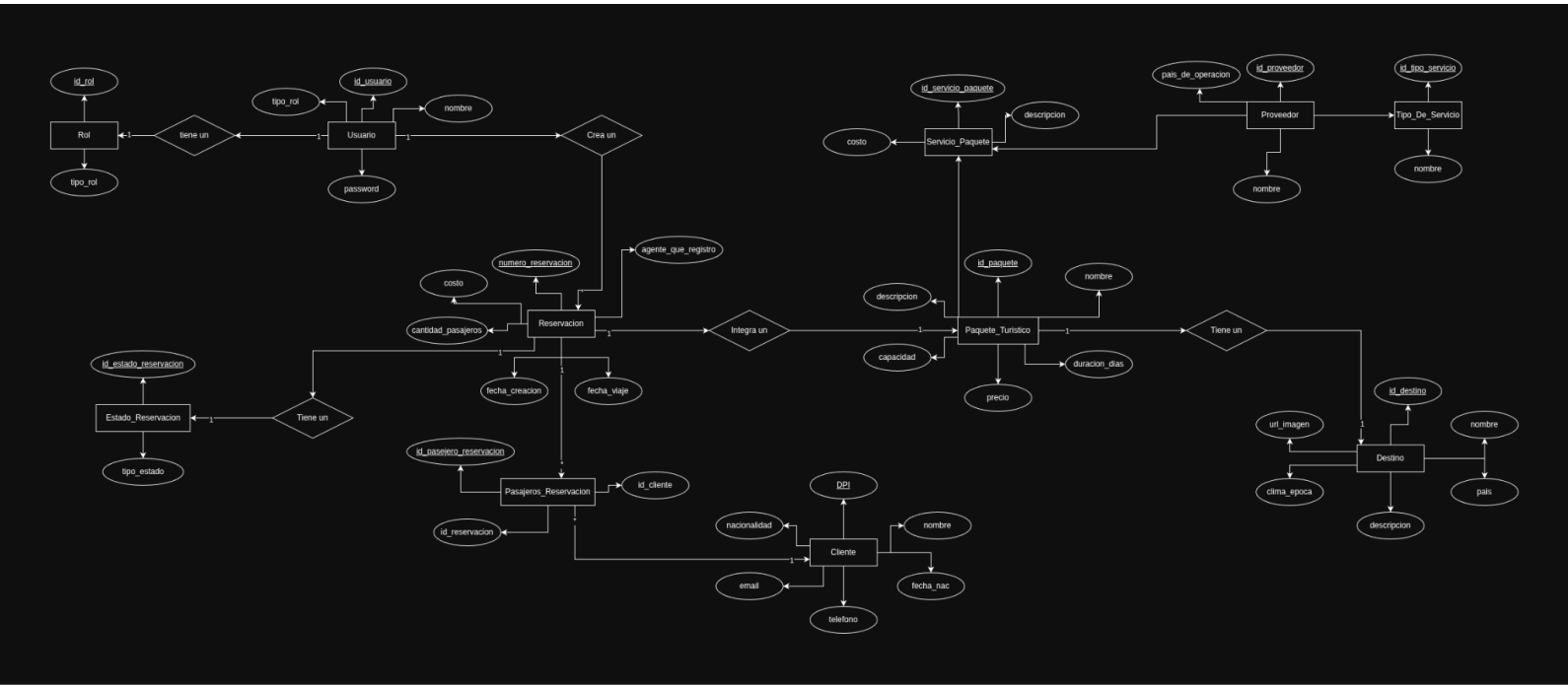
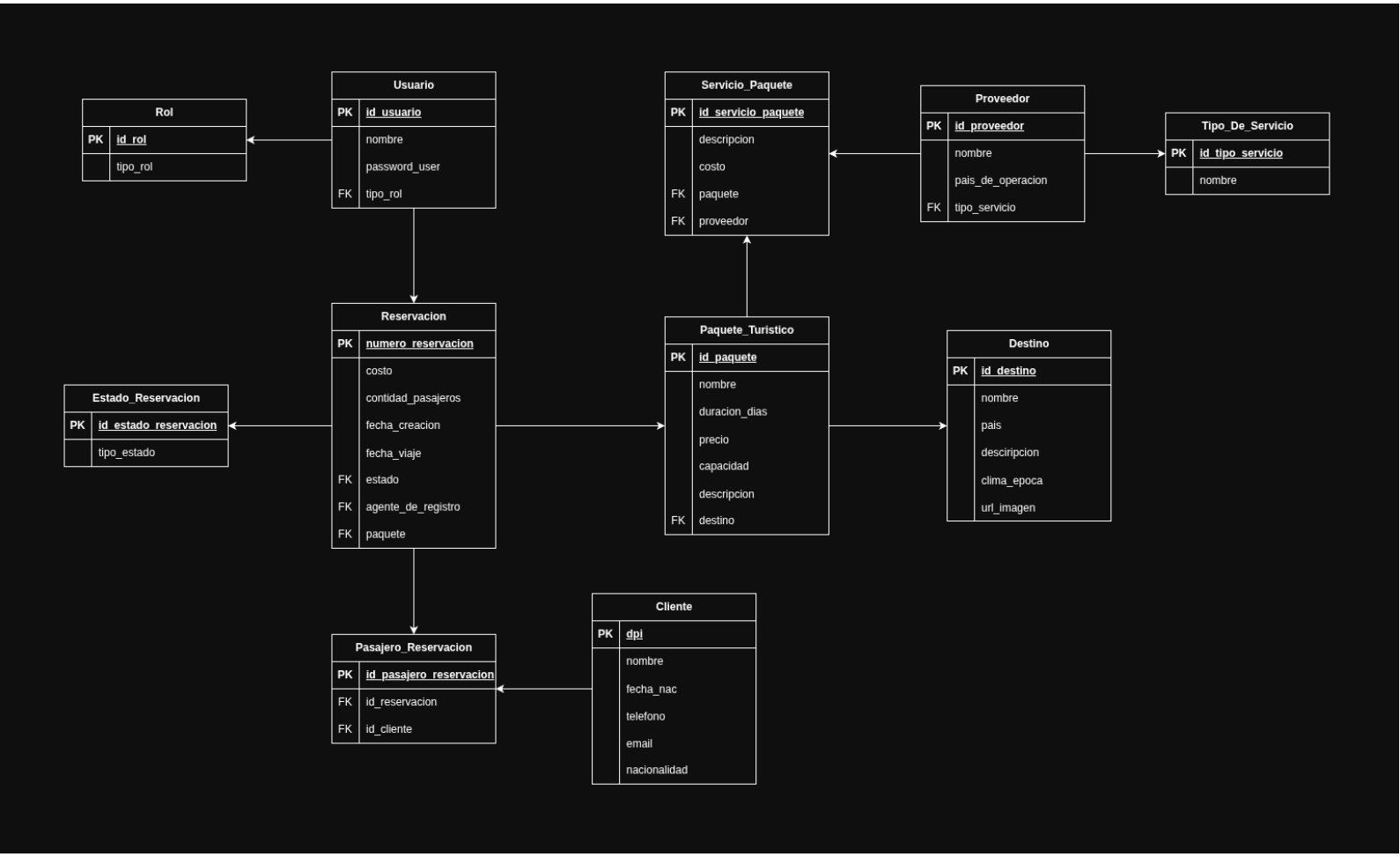


Diagrama de Tablas



Manual Técnico

Descripción General de Clases Importantes

Servlets de Java	Son los encargados de interactuar con la base de datos, la aplicación web manda peticiones para buscar elementos o insertar datos y los servlets reciben esta información y hacen lo pertinente
DAOs del sql	Son las clases encargadas de hacer las consultas con la base de datos, estas clases buscan y arman las listas con los elementos solicitados
ArchivosServlet	Encargado de recibir archivos, dirigirlos al filtro para su construcción e inserción en la base de datos
PagoPDF	Encargado de crear un archivo pdf usando las librerías correspondientes
Angular	Ordena la información recibida desde el backend

1. Servlets de Java

Estos servlets son peticiones http recibidas desde, en este caso, de angular el cual manda peticiones de tipo get, post, put o delete según como se quieran manipular los objetos de la base de datos.

1.1. Peticiones de tipo Get

La petición de tipo get se hace de forma que tiene que recibir en que se espera que devuelva un valor o un arreglo de valores, esta petición no permite enviar un objeto cómo body, en su lugar, se opta por mandar instrucciones en la misma petición http, concatenando elementos string separando con “/”, y en el backend hay que encargarse de filtrar que es lo que se quiere, por ejemplo:

```
public getReservacionesEntreFechas(fechaInicial: string, fechaFinal: string): Observable<Reservacion[]> {
```

```
return this.htttpClient.get<Reservacion[]>
(`${this.restApi.getApiURL()}reporte/entre-fechas/${fechaInicial}/${fechaFinal}`);
```

Esta es una petición que se hace al servlet de reporte, mando una instrucción para manejar ese caso “entre-fechas” y después se procede a mandar un par de filtros, que en este caso son fechas pasadas como string.

En la parte del backen pasa los siguiente:

Primero se procede a recibir la información que viene de la peticición

```
String path = request.getPathInfo();
```

Si el path recibido no es nulo o si no solamente trae la diagonal, quiere decir que efectivamente trae información que puede procesarse, acá si la petición tiene escrito entre-fechas, entonces ya se conoce a que if es al que irá dirigido.

```
if (path != null && !path.equals("/")) {
        String[] partes = path.substring(1).split("/");
        if (partes[0].equals("entre-fechas")) {
            String fechaInicial = partes[1];
            String fechaFinal = partes[2];

                List<ReservacionDB> reservaciones =
reportedao.getReservacionesEntreFechas(fechaInicial, fechaFinal);
            response.getWriter().write(gson.toJson(reservaciones));
```

Acá espera específicamente 2 campos los cuales son fechas porque en esta petición tiene la intención de traer las reservaciones entre un rango X de fechas.

En la parte final se utiliza la lista de ReservacionDB y se escribe en un formato json, importante, los nombres que tiene la clase de Reservacion, deben coincidir con nombres del objeto que se supone recibirá los datos en el frontent, estos objetos son escritos en el objeto HttpServletResponse con ayuda de un objeto Gson que los transforma a un formato json legible para typescript.

1.2. Peticiones de tipo Post

Pasando a los post, estos solo entran a la petición Servlet de tipo Post, no es común mandar algo por la petición pero tiene la diferencias que espera un elemento como body, estos objetos por lo general no esperan algún retorno asi que solo esperan void

```
public crearReservacion(nuevo: Reservacion): Observable<void> {
  return this.httpClient.post<void> (`${this.restApi.getApiURL()}reservacion`, nuevo);
```

```
}
```

Este tipo de petición tiene su diferencia en el backend, ya que el objeto de tipo json enviado desde el frontent, es transformado a un objeto de java apoyándose de un objeto Gson, de igual forma los nombres que vienen desde el frontent deben de coincidir exactamente con los nombres del objeto de java, si no estos no serán leídos por el método.

```
PagoRequest pagorequest = gson.fromJson(request.getReader(), PagoRequest.class);
```

1.3. Peticiones de tipo Put

Las peticiones de tipo put funcionan como una combinación de los dos anteriores, por lo general se manda un complemento en el path de la petición como el id del objeto a manipular, junto a un objeto que serán los valores que se pretenden cambiar o actualizar dentro de la base de datos.

```
public actualizarReservacion(actual: Reservacion): Observable<void> {
  return this.httpClient.put<void> (`${this.restApi.getApiURL()}reservacion`, actual);
}
```

1.4. Peticiones de tipo Delete

Las peticiones de tipo delete son aún más sencillas, ya que solo sería necesario mandar un id dentro del path, y con una consulta de tipo DELETE eliminar el objeto que coincida con el id

```
public eliminarPaquete(idPaquete: number): Observable<void> {
return this.httpClient.delete<void> (`${this.restApi.getApiURL()}paquete/${idPaquete}`);
}
```

Si hablamos de la aplicación del Proyecto 1 de IPC2, puede hacer todas estas opciones con casi todos los objetos, salvo puntos contados que no tendrían sentido afectarlos de esa forma.

1.5. Angular y su uso de los FormGrop y FormControl

Una muy buena utilidad de angular, es que utiliza objetos que funcionan de manera reactiva, es decir, que se actualizan en tiempo real cuando el usuario interactúa con ciertos elementos, por ejemplo un input, si el input está seleccionado por un FormControl, cuando en el input se escriba algo, el formcontrol reacciona y guarda inmediatamente el valor en un atributo, asi no es necesario actualizarlo en cada ocasión con un botón y el FormGroup es un objeto que almacena objetos de FormControl, esto facilita la obtención de objetos y su guardado porque al obtener los valores con el método .value, se obtienen en formato json, asi que en ocasiones ya ni siquiera es necesario transformarlo a un objeto de un modelo de objeto guardado.

1.6. Login

Al inicio del programa, lo que se encuentra es un login, este hace uso de un FormGrop para obtener los valores del usuario posteriormente hace una petición de tipo post, porque de ser un Usuario de Atención al cliente, este debe de registrar el id del usuario porque es un campo necesario para la construcción de una reservación que es utilizado al momento de reportes

1.7. Registro de Clientes

Esta es una opción del usuario de atención al cliente, dentro de sus opciones está la de registrar nuevos clientes, la forma en que se guarda los valores de los campos es usando un GroupForm y procede a mandar la información por un post, el sistema permite usar peticiones de Get para mostrar los Clientes guardados, put al momento de actualizar su información o de tipo put si lo que se quiere es eliminar al cliente de la base de datos.

1.8. Reservaciones

Esta es una opción del usuario de atención al cliente, dentro de sus opciones esta la de crear una nueva reservación llenando los campos que se pide, al momento de guardar la reservación utiliza una petición de tipo post, se pueden actualizar los valores con una petición de tipo put o traer las reservaciones con get, este tiene varias opciones con el get, en ocasiones por fechas o por destino o según lo que se quiera visualizar, en este caso no se agregó la opción de eliminar una reservación porque no tiene sentido que se elimine porque se esos valores sirven para obtener información.

1. 9. Pagos

Esta también es una opción de atención al cliente, este genera pagos con post, aunque este tiene la peculiaridad de que genera y devuelve documentos en pdf para ser descargados. Los pdf no se pueden trabajar como archivos json porque son objetos binarios, se reciben como Blob y usa el siguiente método

```
descargarPDF(): void {
  const url = window.URL.createObjectURL(this.pdfDePago()!);

  const a = document.createElement('a');
  a.href = url;
  a.download = "Recibo de pago";
  a.click();

  window.URL.revokeObjectURL(url);
}
```

la primera línea crea un url, con window crea un objeto utilizando la base de la url donde se está en ese momento, haciendo algo como esto
blob:http://localhost:4200/atencion-cliente

como los navegadores solo permiten descargar mediante un enlace y se crea una etiqueta de tipo enlace en el que se hace click con código y no de manera manual, el download hace que en lugar de abrir el enlace, solo descargue lo que tiene el enlace y algo importante es cerrar el objeto window.URL para evitar fugas de memoria

1.10. Subida de Archivos

El sistema permite subir archivos para cargar al sistema, en la parte de Angular se utiliza:

```
<input type="file" (change)="cargarArchivo($event)">
```

Por la parte del HTML se utiliza una input de tipo file y utilizando al “(change)” se detecta cuando sucede un cambio en el input, en este caso cuando detecta un archivo, lanza el método mandando el archivo por parámetro.

```
cargarArchivo(event: any): void {
  this.archivoElegido = event.target.files[0];

  const formData: FormData = new FormData();
  formData.append('archivo', this.archivoElegido);
```

```
  this.cargarArchivosService.cargarArchivo(formData).subscribe({
    next: (errores: RegistroErrores) => {
      alert('El archivo ha sido cargado exitosamente');
      this.registroErrores.set(errores);
    },
    error: () => alert('Ocurrio un problema con el Archivo')
  })
}
```

En la parte del typescript se obtiene ese evento y se transforma a un archivo listo para ser enviado al backend, al inicio el event.target atrapa el input de tipo file que es enviado desde el HTML, el .files pasa el input a un arreglo de archivos por eso se escoge el elemento 0, porque ahi es donde estará el archivo file.

De igual forma que antes, el archivo no puede pasarse como json, sino que se manda como binario y para eso se tiene que atrapar con un objeto de FormData y para agregar un elemento se usa el método de .append, junto con una clave que es la que buscará el backend así que debe ser igual en el backend.